
APS360 Progress Report

He Ming (Andy) Liu

1010205109, heming.liu@mail.utoronto.ca

<https://github.com/liuandy1207/BreadCrumb/tree/main>

Introduction

Prototyping electronic circuits on breadboards is common practice in engineering labs and electronics projects, but manually documenting the components used in a circuit can be tedious and error-prone. The goal of this project, *BreadCrumb*, is to develop a system that automatically generates a bill of materials (BoM) from a photograph of a breadboard circuit. Given an image of a breadboard containing electronic components, the model identifies each component type to produce a structured list of component types and their counts. This tool streamlines documentation and material tracking during prototyping, making it easier for engineers, students, and hobbyists to record and reproduce circuits efficiently. Deep learning is a suitable approach for this task because convolutional neural networks (CNNs) learn hierarchical visual features, allowing them to recognize and distinguish circuit components directly from images. See Figure 1 for details.

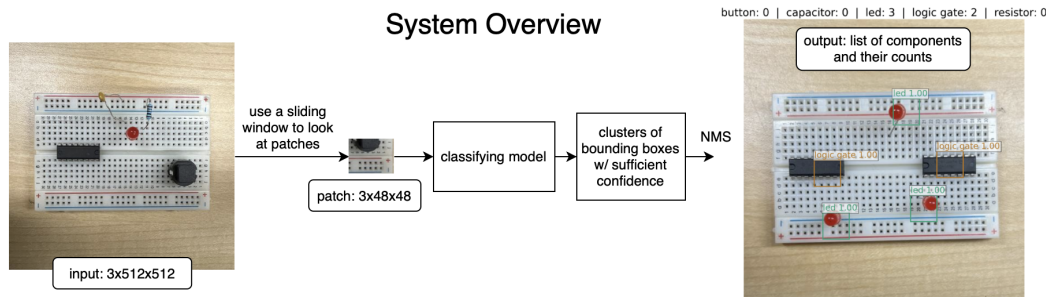


Figure 1: System overview. The input is a 512×512 RGB image of a breadboard circuit. This image is scanned with a sliding 48×48 pixel window (stride 16). Each window is classified by a CNN-based classifier into one of six categories (background, button, capacitor, LED, logic gate, or resistor). Windows containing a component are recorded as bounding boxes. Non-maximum suppression (NMS) collapses duplicate detections into a single highest-confidence box per component. The surviving boxes are counted by class to produce a structured list of component types and their counts.

Background & Related Work

Prior work addresses circuit recognition and schematic analysis. The Resistor Detection Model focuses on identifying resistors on breadboards [2]. Circulens converts breadboard images into circuit diagrams [3]. ESR generates SPICE models from schematics [5], while SINA produces netlists using vision and OCR methods [1]. Circuitry.ai combines vision and language models for schematic understanding [6]. These approaches focus on schematics or single component types. In contrast, this project performs multi-class detection directly on breadboard images to generate component counts.

Data Processing

Training data was collected by constructing circuits on half-sized breadboards using resistors, LEDs, buttons, logic gates, and capacitors. Images were captured top-down under consistent lighting, with components clearly visible and unobstructed. Images were framed to include only the breadboard (and the background table). A total of 100 images were uploaded to Roboflow, where bounding boxes were manually drawn around each component and labelled by class. Images were resized to 512×512 , split 80/10/10 for training, validation, and testing, and exported in COCO JSON format. During training, Each image is processed using a 48×48 sliding window with stride 16 to generate overlapping patches. Patches are labeled by the component whose bounding box center they contain, or background otherwise, and normalized using ImageNet statistics. Dataset statistics are shown in Table 1 and Figure 2 shows an example dataset image. The quantitative and qualitative results presented later are based on the original 10-image held-out test split that was never seen during

training or validation, while additional test data comprising of 10 images was collected separately for evaluation in the "Evaluating on New Data" section.

Table 1: Component instances per class across 100 dataset images.

Component	Instances
Resistors	77
Red LEDs	89
Capacitors	48
Push Buttons	56
Logic Gates	50

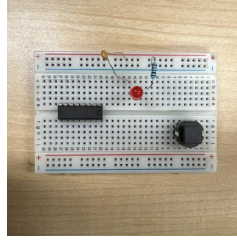


Figure 2: Example dataset image (RGB 512×512 px) showing all five component classes.

Architecture

The primary model uses ResNet18 pretrained on ImageNet as a frozen feature extractor (11,689,512 parameters), with a trainable two-layer classification head (132,870 trainable parameters) [4]. This architecture is well-suited for the task because ResNet18’s pretrained convolutional features generalize effectively to component recognition even with a small dataset. The model takes 48×48 RGB patches as input and outputs class probabilities over six categories: background, button, capacitor, LED, logic gate, and resistor. The custom head consists of a linear layer (512 input neurons to 256 hidden neurons) with ReLU activation and 30% dropout, followed by a final linear layer (256 hidden neurons to 6 output classes). During inference, a sliding window with stride 16 scans the 512×512 input image, generating patch predictions. Predictions with confidence above 0.99 (excluding background) are converted to bounding boxes, and non-maximum suppression with IoU threshold 0.0001 removes duplicate detections. See Figure 7 in the Appendix for an architecture diagram.

Baseline

The baseline model is a fully-connected artificial neural network (ANN) serving as a simple reference to evaluate the benefit of transfer learning in the primary model. The architecture consists of three linear layers with ReLU activations and dropout. The input is a flattened $48 \times 48 \times 3$ patch (6,912 values), which passes through a linear layer (6,912 input neurons to 256 hidden neurons), ReLU activation, 30% dropout, a second linear layer (256 hidden neurons to 128 hidden neurons), ReLU activation, and a final linear layer (128 hidden neurons to 6 output classes). The model contains 1,803,398 trainable parameters. It does not rely on pretrained weights and uses a fixed, lightweight architecture without extensive tuning, allowing it to be trained quickly from scratch and serve as a straightforward baseline. This simple architecture provides a clear benchmark to measure whether ResNet18’s pretrained features meaningfully improve component patch classification performance. See Figure 8 in the Appendix for an architecture diagram.

Quantitative Results

The results presented in this section and the section below are based on the original 10-image test split that was never seen during training or validation. Per-class mean absolute error (MAE) was chosen as the primary metric of performance because the end goal is to generate accurate material lists. MAE directly measures counting accuracy as the average difference between predicted and actual component counts per image. Thus, a lower MAE means more reliable bills of materials. Patch classification accuracies and losses are available in the Appendix (Figure 9 and Figure 10), but correct components counts matter more for the context of the problem. Overall, the primary model reduces counting errors by 72% compared to baseline. However, the small test set size (10 images) limits statistical reliability, and results may not fully reflect performance across diverse breadboard configurations.

Qualitative Results

Failure mode analysis reveals several consistent challenges. First, the primary model struggles to merge multiple detections into a single bounding box for larger components such as logic gates, leading to overcounting (Figure 4). This likely occurs because logic gates span multiple patches. With the given patch size, detections at opposite ends may not overlap sufficiently (zero IoU) to be merged. This highlights a key limitation of patch based detection, where lack of global context makes it difficult to associate multiple detections with a single object. In contrast, the baseline model frequently misclassifies logic gates as buttons or fails to detect them altogether (Figure 5), explaining the inflated button counts and undercounted logic gates in Figure 3. This behaviour likely stems from lower classification confidence as bounding box generation requires high-confidence predictions. This could be a result of the simpler fully connected ANN lacking the capacity to extract robust spatial features compared to the transfer-learned CNN. Capacitors present an additional challenge

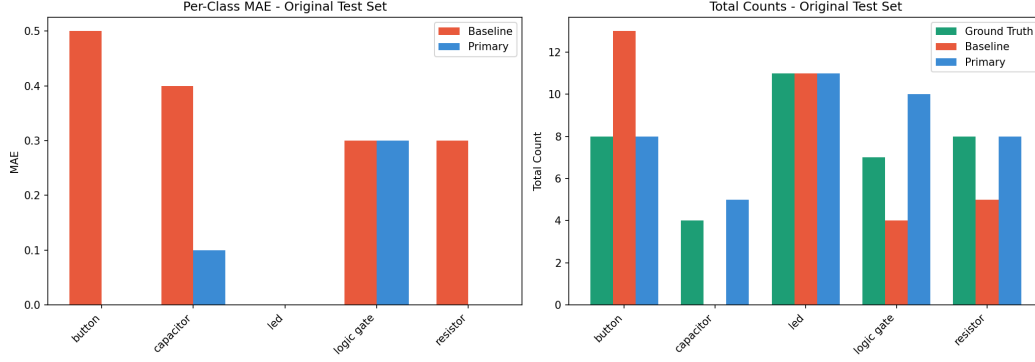


Figure 3: Left: Per-class MAE on the test set. The primary model achieves zero error on buttons, LEDs, and resistors, demonstrating strong performance on components of moderate size. The baseline struggles with buttons and resistors, but matches primary’s performance for LEDs, likely due to their distinct appearance. Both models struggle equally with logic gates (MAE = 0.30), but baseline struggles significantly more to count capacitors for reasons explored in the qualitative results. Right: Total counts across the 10-image test set. The primary model achieves exact matches on buttons (8/8), resistors (8/8), and LEDs (11/11). However, it overcounts logic gates (10 predicted vs. 7 actual) and capacitors (5 vs. 4). The baseline model overcounts most components (except LEDs) and completely fails to count any capacitors.

due to their small size and low visual contrast, making them difficult to detect. Notably, the baseline fails to identify any instances. In contrast, visually distinctive components such as red LEDs are consistently classified correctly by both models.

button: 1 | capacitor: 0 | led: 1 | logic gate: 2 | resistor: 1

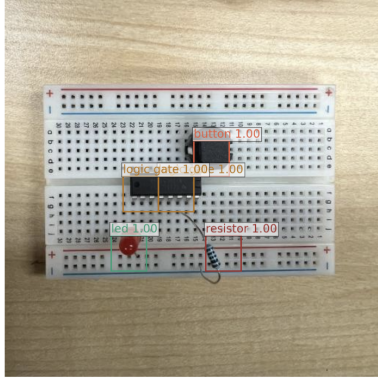


Figure 4: Example of primary model output for a test image. The model fails to combine the bounding boxes for a logic gate, resulting in an incorrect total.

Evaluating on New Data

To evaluate generalization, ten additional images were collected using a different camera while maintaining the same setup and preprocessing pipeline. The primary model maintains low counting error on unseen data, with total MAE of 0.24 compared to baseline’s 0.7. While per-class errors increased for buttons and resistors relative to the original test set, the model still produces more accurate overall counts than the baseline, which continues to fail entirely on capacitors and struggles with resistors. The performance degradation on specific classes is consistent with domain shift without fine-tuning, and demonstrates that the core learned features generalize adequately across imaging conditions, and could improve with more diverse training data or camera-specific fine tuning.

Discussion

The model performs well on most components, suggesting that combining CNN-based patch classification with a sliding window approach is a viable strategy for multi-class counting, but struggles with larger or less distinct classes such as logic gates. The main limitation arises in post-processing rather than classification. While patch classification accuracy reaches 97.7% as shown in the Appendix in Figure 9, this does not directly translate to accurate component counts. Errors are primarily caused

button: 1 | capacitor: 0 | led: 3 | logic gate: 0 | resistor: 0

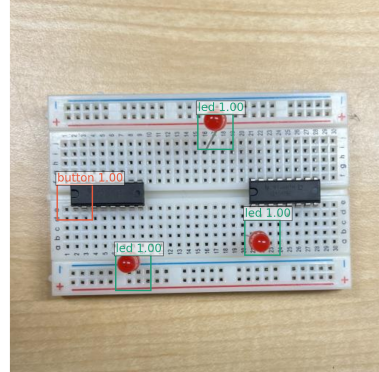


Figure 5: Example of baseline output for a test image. The model mistakes one logic gate for a button and misses another, but successfully locates the LEDs.

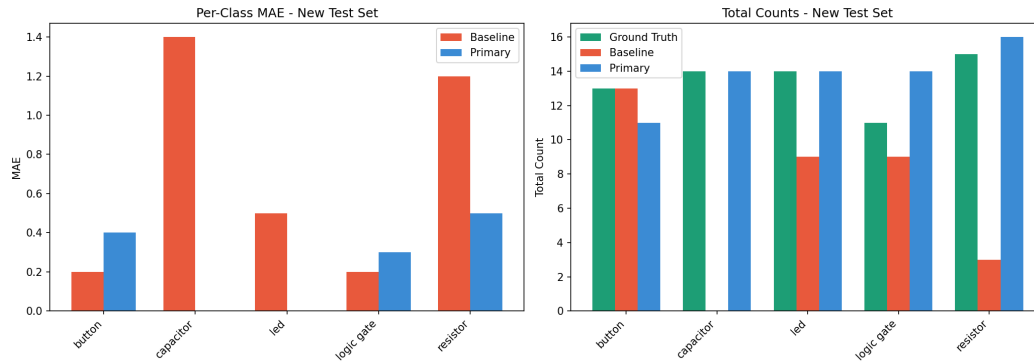


Figure 6: Left: Per-class MAE on the new test set. The primary model exhibits increased error for buttons and resistors compared to the baseline, in contrast to its performance on the original test set. Right: Total predicted vs. actual counts across the 10-image test set. The baseline generally underestimates component counts, while the primary model remains relatively accurate, with errors in resistor and button counts.

by incomplete merging of bounding boxes, especially for components that span multiple patches. The gap between model metrics and task performance became clear during this project. Overall performance is driven more by post-processing than classification quality, highlighting that strong intermediate metrics can be misleading in an end-to-end system. This suggests that improving the classifier alone would not significantly improve end-to-end performance. Furthermore, this project also showed how concepts from class such as CNN architectures and transfer learning extend into a full pipeline where classification is only one part. Manually annotating 100 images highlighted the difficulty of data collection and labeling as a major bottleneck in machine learning workflows. Finally, tuning the NMS IoU threshold showed how sensitive overall performance is to post-processing hyperparameters, reinforcing that effective systems require optimization across the entire pipeline, not just the neural network.

Ethical Considerations

One potential ethical concern arises if the model misidentifies or miscounts components, leading users to build faulty or potentially dangerous circuits. The model also has inherent limitations. Training data consists of neatly arranged breadboards photographed under consistent lighting conditions with no wires present, so performance may degrade with messy circuits, wired circuits, or poor lighting. The dataset covers only five common component types, and the model classifies components into broad categories without identifying specific values like resistance, capacitance, or logic gate type. Users relying on the system for complete documentation may overlook unlabeled components or assume specificity that the model cannot provide, leading to incomplete / inaccurate bills of materials. Additionally, the controlled dataset introduces bias toward clean layouts, which may lead to overconfidence when applied to more realistic or cluttered circuits.

Project Difficulty

This project went beyond typical lab scope by requiring multi-object detection instead of single-image classification, as well as the design of a complete end to end pipeline from data collection to post-processing. Unlike standard labs, where datasets and evaluation frameworks are provided, this work involved developing the data collection process, annotation workflow, and task specific evaluation methods from scratch. It also required learning and applying techniques such as sliding window detection and non-maximum suppression that were not covered in the course. As a result, the problem was more challenging than it may initially appear. Multi component classification differs fundamentally from the single object setting used in labs, particularly in how predictions must be generated across an entire image. A key challenge was designing a method to scan images and detect multiple components simultaneously. Several approaches were explored before settling on a sliding window strategy with patch classification, combined with non-maximum suppression for post-processing. This required careful tuning of IoU thresholds, confidence levels, window size, and stride to ensure correct merging and accurate counts. More importantly, it highlighted that multi object detection introduces challenges beyond classification, particularly in aggregating predictions across an image. The final system demonstrates that course concepts can be extended to this setting, while also revealing limitations in patch based detection and post-processing.

References

- [1] Saoud Aldowaish, Yashwanth Karumanchi, Kai-Chen Chiang, Soroosh Noorzad, and Morteza Fayazi. Sina: A circuit schematic image-to-netlist generator using artificial intelligence. *arXiv preprint arXiv:2601.22114*, 2026.
- [2] Circuits Project. Resistor detection dataset. <https://universe.roboflow.com/circuits-project/resistor-detection-5azes>, 2023.
- [3] Ibrahim Ahmed Mohammed. Circulens: Ai breadboard to diagram tool. <https://devpost.com/software/ciculens>, 2025.
- [4] PyTorch Contributors. resnet18 - torchvision documentation. <https://docs.pytorch.org/vision/main/models/generated/torchvision.models.resnet18.html>, 2025. Accessed: 2026-03-13.
- [5] tinkrmind. Electronic schematic recognizer. <https://github.com/tinkrmind/electrical-schematic-recognizer>, 2016.
- [6] tonny-2200. Circuitry.ai: Ai-assisted circuit diagram analyzer. <https://github.com/tonny-2200/circuitry>, 2025.

Appendix

These materials are supplementary to the main four pages of text.

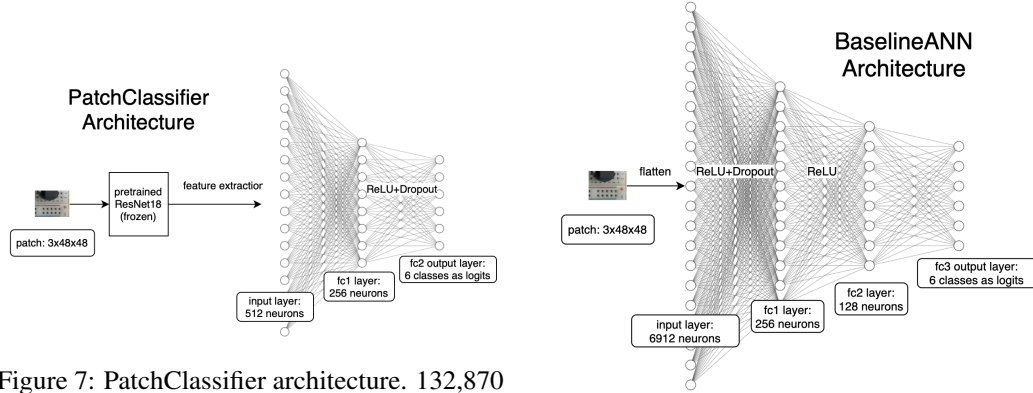


Figure 7: PatchClassifier architecture. 132,870 trainable parameters. 11,689,512 frozen parameters from ResNet18 [4].

Figure 8: BaselineANN architecture. 1,803,398 trainable parameters across 3 fully-connected layers.

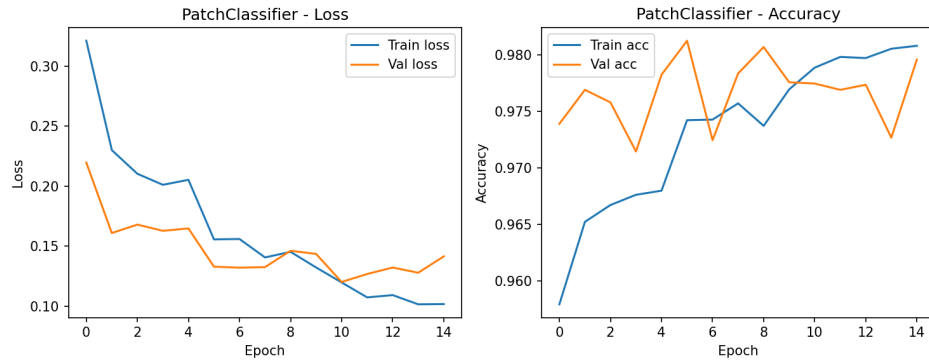


Figure 9: PatchClassifier learning curves. Best train loss: 0.1199, train accuracy: 0.979. Best validation loss: 0.1203, validation accuracy: 0.977. Accuracy is computed over component patches only, excluding background. Validation accuracy exceeding training accuracy suggests the model generalizes well and is not overfitting.

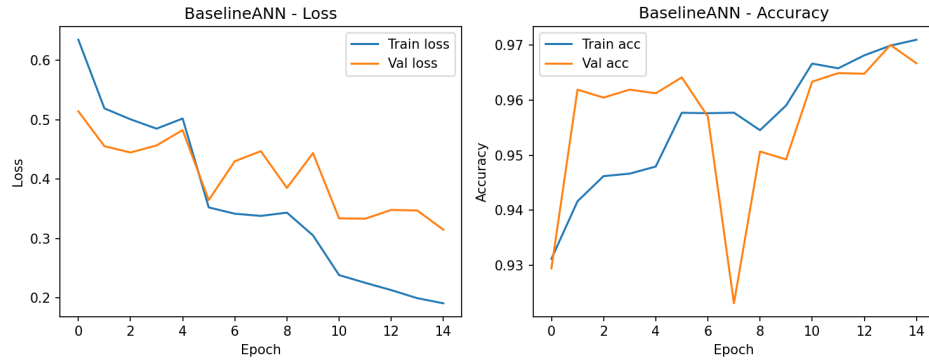


Figure 10: BaselineANN learning curves. Best train loss: 0.1908, train accuracy: 0.971. Best validation loss: 0.3147, validation accuracy: 0.967. Accuracy is computed over component patches only, excluding background.